Chapter 2: Pandas: DataFrames

<{http://www.bookeducator.com/Textbook}learningobjective >Students will be able to construct Pandas DataFrames from multiple data sources including dictionaries, lists, and external files
<{http://www.bookeducator.com/Textbook}learningobjective >Students will be able to access DataFrame elements using label-based (.loc) and position-based (.iloc) indexing for columns, rows, and individual cells
<{http://www.bookeducator.com/Textbook}learningobjective >Students will be able to add new rows and columns to DataFrames using appropriate methods including direct assignment, .insert(), .join(), and .merge()
<{http://www.bookeducator.com/Textbook}learningobjective >Students will be able to distinguish between index-based joins (.join()) and key-based merges (.merge()) for combining DataFrames

2.1Introduction

The Pandas DataFrame A two-dimensional, size-mutable, potentially heterogeneous tabular data structure with labeled axes (rows and columns). is the foundational data structure used throughout modern Python-based data analysis. Designed to organize and manipulate tabular data, DataFrames

support efficient analysis, transformation, and extraction of insights, making them central to Python's dominance in data science and machine learning.

While Python provides many data structures, DataFrames are uniquely suited for real-world data projects. Lists are one-dimensional and position-based, dictionaries store key-value pairs without a natural tabular structure, and NumPy arrays require homogeneous data types. In contrast, DataFrames allow columns of different data types, labeled indexing, and column-wise operations that closely match how data is stored and analyzed in practice.

The figure below shows how a DataFrame is typically displayed in a Jupyter Notebook (.ipynb) environment:

name	age	quote
Homer	38	Operator! Give Me The Number For 911!
Marge	38	Bart, don't make fun of grad students. They just made a terrible life choice.
Bart	10	I'd say that the pressure's finally gotten to dad, but, what pressure?
Lisa	8	Why do I get the feeling that someday I'll be describing this to a psychiatrist?
Maggie	1	[sucking sound]

Conceptually, a DataFrame can be thought of as an in-memory spreadsheet with powerful programmatic capabilities. Unlike traditional spreadsheets, DataFrames support vectorized operations, integration with statistical and machine learning libraries, and seamless scaling from small exploratory analyses to large production workflows.

At a technical level, DataFrames are created using a constructor A special type of subroutine called to create an object. It prepares the new object for use, often accepting arguments that the constructor uses to set required member variables.. You do not need to memorize the constructor signature at this

stage; instead, think of it as a preview of the flexibility Pandas provides when creating tabular data structures.

Pandas DataFrame(): documentation
DataFrame([data, index, columns, dtype, copy])

- **data**: the tabular data itself, provided as a dictionary, list, Pandas Series, or other array-like structures
- **index**: labels for each row; defaults to an integer-based index if not specified
- **columns**: labels for each column; defaults to an integer-based index if not specified
- **dtype**: the data type assigned to columns; if provided, it must be compatible with the underlying data
- **copy**: whether the DataFrame should be a copy of the input data or a reference to it

In practice, many of these parameters are optional, and Pandas will infer reasonable defaults. As you work through this chapter, you will see concrete examples of creating DataFrames in different ways and learn when it makes sense to explicitly control these options.

Why Learn DataFrames?

DataFrames form the backbone of nearly every phase of a data project after data collection. In the CRISP-DM framework introduced earlier in this book, DataFrames are the primary structure used for data understanding, preparation, modeling, evaluation, and deployment. Mastery of DataFrames directly translates into greater efficiency, clarity, and confidence when working on real-world data problems.

This chapter introduces the core skills needed to create, read, modify, filter, and sort DataFrames. These skills will be used repeatedly throughout the remainder of the book, serving as the foundation for data wrangling, exploratory analysis, and machine learning workflows.

2.2Creating DataFrames

In this section, you will create DataFrames using several common patterns. You do not need to memorize each pattern; instead, focus on understanding how the *data*, *columns*, and *index* work together. Once those relationships are clear, you can choose the most convenient approach for a given situation.

The most common way to create a DataFrame is from a Python dictionary of lists. Recall that when you import a library, you typically assign it a short alias (for example, *pd* for Pandas). The alias name is a variable, and you can technically name it anything, but *pd* is the widely used convention. To call the DataFrame constructor, you reference *pd* as shown below:

```
# Import the pandas package
import pandas as pd

# Create a dictionary where each key becomes a column and each list contains the
column values
heart_rates = {
  'participant': ['p1', 'p2', 'p3', 'p4'],
  'hr1': [98.1, 78.0, 65.0, 64.0],
  'hr2': [110, 120, 129, 141],
  'hr3': [76, 87, 77, 59]
}

# Construct a DataFrame from the dictionary
df = pd.DataFrame(heart_rates)
df
```

To view the DataFrame, you can use the *print()* function:

```
print(df)
```

```
  participant   hr1  hr2  hr3
0          p1  98.1  110   76
1          p2  78.0  120   87
2          p3  65.0  129   77
3          p4  64.0  141   59
```

When you use *print(df)*, Pandas displays a plain-text representation of the table. In Jupyter and Colab notebooks, there is a second viewing option: if the last line of a code cell is a DataFrame (or many other Pandas objects), the notebook will automatically render a nicely formatted table. To see that version, evaluate *df* without using *print()*:

```
df
```

	participant	hr1	hr2	hr3
0	p1	98.1	110	76
1	p2	78.0	120	87
2	p3	65.0	129	77
3	p4	64.0	141	59

That looks a bit better. In a notebook, this nicely formatted output appears only for the final expression in a code cell. If you display something else afterward (including using *print()*), the notebook will show the later output

instead. For example, the formatted table does not appear below because the cell ends with *print(df)*:

```
df  # This would render as a formatted table if it were the last line in the cell
print(df)  # This prints a plain-text version


# Output
# participant   hr1  hr2  hr3
# 0           p1  98.1  110   76
# 1           p2  78.0  120   87
# 2           p3  65.0  129   77
# 3           p4  64.0  141   59
```

Notice a few details. First, notice the unlabeled column of numbers from 0 to 3 on the left. This is the default row index called a *RangeIndex*. Pandas creates it automatically when you do not specify a custom index.

Second, the column names came from the dictionary keys (*participant*, *hr1*, *hr2*, and *hr3*). These are the labels for the column axis. Internally, Pandas also maintains positional indexes, but when a label exists, Pandas displays the labels because they are easier for people to read.

In summary, a DataFrame has two axes: rows and columns. Each axis has labels. If you do not supply a custom row index, Pandas uses a *RangeIndex* (0, 1, 2, ...) as the row labels. Column labels, however, must always exist because every column needs a name.

column ranged index

	0	1	2	3	4	5	6	7	
row ranged index	**id**	**age**	**sex**	**bmi**	**children**	**smoker**	**region**	**charges**	<- labeled index
0	**C001**	19	female	27.9	0	yes	southwest	16884.924	
1	**C002**	18	male	33.77	1	no	southeast	1725.5523	
2	**C003**	28	male	33	3	no	southeast	4449.462	
3	**C004**	33	male	22.71	0	no	northwest	21984.4706	
4	**C005**	32	male	28.88	0	no	northwest	3866.8552	
5	**C006**	31	female	25.74	0	no	southeast	3756.6216	
6	**C007**	46	female	33.44	1	no	southeast	8240.5896	
7	**C008**	37	female	27.74	3	no	northwest	7281.5056	
8	**C009**	37	male	29.83	2	no	northeast	6406.4107	
9	**C010**	60	female	25.84	0	no	northwest	28923.1369	

labeled index ->

In the first constructor example, we provided only the data. The row index, column labels, and data types were inferred automatically. You can inspect what Pandas inferred by checking the DataFrame's *index*, *columns*, and *dtypes*:

```
print(df.index)
print()
print(df.columns)
print()
print(df.dtypes)


# Output
# RangeIndex(start=0, stop=4, step=1)
#
# Index(['participant', 'hr1', 'hr2', 'hr3'], dtype='object')
#
# participant     object
# hr1            float64
# hr2              int64
# hr3              int64
# dtype: object
```

Notice that *df.index* is the *RangeIndex* for the rows because we did not specify a custom row index. The *df.columns* attribute contains the column labels inferred from the dictionary keys. Finally, *df.dtypes* shows the data type inferred for each column. In this example, *participant* is *object* because it stores general Python objects (most commonly strings), and the heart rate columns are numeric types. You will later learn how to explicitly convert types when needed.

You can also create DataFrames using the constructor more directly. Here is an empty DataFrame with the columns specified up front:

```
import pandas as pd

df = pd.DataFrame(columns=['participant', 'hr1', 'hr2', 'hr3'])
df = df.set_index('participant')
df
```

	hr1	hr2	hr3
participant			

Empty DataFrames are useful when you plan to build a results table step-by-step, such as when iterating through another dataset and calculating summary statistics. You define the structure first, then add rows as you compute results.

To create a DataFrame with data, pass a list of rows (a list of lists) to the constructor:

```
import pandas as pd

df = pd.DataFrame(
  data=[[98.1, 110, 76], [78.0, 120, 87], [65.0, 129, 77], [64.0, 141, 59]],
  index=['p1', 'p2', 'p3', 'p4'],
  columns=['hr1', 'hr2', 'hr3']
)

df
```

	hr1	hr2	hr3
p1	98.1	110	76
p2	78.0	120	87
p3	65.0	129	77
p4	64.0	141	59

Notice that the column name *participant* is not shown because the participant identifiers are now used as the row index labels. Although a *RangeIndex* still exists internally, it is not displayed because the DataFrame now has labeled row indexes. This example is also a good reminder of a key distinction: dictionaries are typically column-oriented (keys map to columns), while a list of lists is often row-oriented (each inner list represents one row).

You can also set the index after a DataFrame has been created. One advantage of this approach is that you can keep the index column as a regular column in the DataFrame until you are ready to set it as the index.

```
import pandas as pd
```

```
df = pd.DataFrame(
  data=[['p1', 98.1, 110, 76], ['p2', 78.0, 120, 87], ['p3', 65.0, 129, 77], ['p4',
64.0, 141, 59]],
  columns=['participant', 'hr1', 'hr2', 'hr3']
)

df = df.set_index('participant')
df
```

	hr1	hr2	hr3
participant			
p1	98.0	110.0	76.0
p2	78.0	120.0	87.0
p3	65.0	129.0	77.0
p4	64.0	141.0	59.0

If you prefer to keep the dictionary form, you can still pass a dictionary into the constructor while specifying the index separately. This approach keeps the data organized by columns while allowing you to define row labels independently:

```
import pandas as pd

df = pd.DataFrame(
  {'hr1': [98.1, 78.0, 65.0, 64.0], 'hr2': [110, 120, 129, 141], 'hr3': [76, 87, 77,
```

```
59]},
        index=['p1', 'p2', 'p3', 'p4']
      )

      df.index.name = 'participant'
      df
```

participant	hr1	hr2	hr3
p1	98.0	110.0	76.0
p2	78.0	120.0	87.0
p3	65.0	129.0	77.0
p4	64.0	141.0	59.0

Each of these creation techniques can be useful depending on what form your data is already in and how you plan to access it.

Why create an index at all? Indexes affect how you locate and align data. For example, many DataFrame operations match rows based on index labels, and indexes can improve performance when filtering and joining. Also note that while a *RangeIndex* is naturally unique, labeled indexes do not have to be unique. In some scenarios, you intentionally use repeated index labels (such as grouping categories) to make certain operations more convenient.

However, when you need unique identifiers, you must ensure uniqueness yourself.

The sections that follow cover how to read, update, and delete information from DataFrames.

2.3Reading DataFrames

Columns

Now that you have a DataFrame, how do you read data from it? The most common way to read a column is to use its column label (name), as shown below with the *hr3* column.

```
# Create a DataFrame (default RangeIndex for rows)
import pandas as pd

df_no_index = pd.DataFrame(
  data=[['p1', 98.1, 110, 76], ['p2', 78.0, 120, 87], ['p3', 65.0, 129, 77],
['p4', 64.0, 141, 59]],
  columns=['participant', 'hr1', 'hr2', 'hr3']
)

print(df_no_index['hr3'])
print()

# Create a DataFrame and set a labeled row index
df_with_index = pd.DataFrame(
  data=[['p1', 98.1, 110, 76], ['p2', 78.0, 120, 87], ['p3', 65.0, 129, 77],
['p4', 64.0, 141, 59]],
  columns=['participant', 'hr1', 'hr2', 'hr3']
).set_index('participant')
print(df_with_index['hr3'])


# Output:
# 0    76
# 1    87
# 2    77
# 3    59
# Name: hr3, dtype: int64
#
# participant
# p1    76
```

```
# p2    87
# p3    77
# p4    59
# Name: hr3, dtype: int64
```

Reading an entire column is as simple as placing the column label in brackets, such as *df['hr3']*. The main difference in the output above is the labeling: with a RangeIndex you see 0, 1, 2, 3, and with a labeled index you see p1, p2, p3, p4. You can also select columns by position, but that is done with *.iloc* (introduced below) rather than by replacing the label with a number.

Rows

Next, select all values of a particular row using the .loc[] A Pandas DataFrame accessor used to select rows and columns by label. and .iloc[] A Pandas DataFrame accessor used to select rows and columns by integer position. accessors. Use *.iloc* for position-based selection and *.loc* for label-based selection.

```
import pandas as pd

df = pd.DataFrame(
  data=[['p1', 98.1, 110, 76], ['p2', 78.0, 120, 87], ['p3', 65.0, 129, 77],
['p4', 64.0, 141, 59]],
  columns=['participant', 'hr1', 'hr2', 'hr3']
).set_index('participant')

# Read a row by position (RangeIndex order)
print(df.iloc[3])

# Read a row by label (Index)
print(df.loc['p4'])


# Output:
# hr1     64.0
# hr2    141.0
# hr3     59.0
# Name: p4, dtype: float64
# hr1     64.0
# hr2    141.0
```

```
# hr3     59.0
# Name: p4, dtype: float64
```

Reading a row is straightforward by either position (*.iloc*) or label (*.loc*). However, label-based selection requires that a labeled row index exists. Also notice that the index label (*participant*) is not printed as a column in the returned row because it is part of the index, not a column.

Pandas Series

When you select a single column or a single row from a DataFrame, the result is typically a Pandas Series A one-dimensional array-like object with axis labels. Available from the Pandas package.. A Series is similar to a one-dimensional Python list, but it includes an index and many useful methods that align with DataFrame behavior. This is important because many operations you perform on DataFrames can also be performed on Series, and you will often see Series objects in examples and troubleshooting discussions online.

Pandas Series()
Series([data=None, index=None, dtype=None, name=None, copy=False]): documentation

- **data**: the values to store (for example, a list, NumPy array, dictionary, or another Series)
- **index**: labels for the values; defaults to a RangeIndex if not provided
- **dtype**: the desired data type; if not provided, Pandas will infer a type
- **name**: an optional label for the Series, often used when the Series comes from a DataFrame column
- **copy**: whether to copy the underlying data (when possible) instead of referencing it

Cells

Finally, you will often need to read a specific cell value (a row and column intersection). A reliable approach is to use *.loc* or *.iloc* and provide both the row reference and the column reference.

```
import pandas as pd

df = pd.DataFrame(
  data=[['p1', 98.1, 110, 76], ['p2', 78.0, 120, 87], ['p3', 65.0, 129, 77],
['p4', 64.0, 141, 59]],
  columns=['participant', 'hr1', 'hr2', 'hr3']
).set_index('participant')

# Return the hr3 value for participant p4 (row label p4, column label hr3)
df.loc['p4', 'hr3']

# Return the cell value by position: 4th row (index 3), 3rd column (index 2) in
the current column order (hr1, hr2, hr3)
df.iloc[3, 2]


# Output: 59
```

It is important to understand why *hr3* is accessed using column position *2*. When the *participant* column is set as the row index, it is removed from the DataFrame's column structure. As a result, the remaining columns are ordered as *hr1* (position 0), *hr2* (position 1), and *hr3* (position 2). The numeric column positions used by *.iloc* always refer to the current column order, not the original column list.

To select an entire column by position (rather than by label), use *.iloc* with *:* for "all rows." The expression *df.iloc[:, 2]* means "all rows" and "the third column." The label-based equivalent is *df.loc[:, 'hr3']*.

```
import pandas as pd

df = pd.DataFrame(
  data=[['p1', 98.1, 110, 76], ['p2', 78.0, 120, 87], ['p3', 65.0, 129, 77],
['p4', 64.0, 141, 59]],
  columns=['participant', 'hr1', 'hr2', 'hr3']
).set_index('participant')
print(df.iloc[:, 2])
print(df.loc[:, 'hr3'])


# Output:
# participant
# p1    76
# p2    87
# p3    77
# p4    59
# Name: hr3, dtype: int64
# participant
# p1    76
# p2    87
# p3    77
# p4    59
# Name: hr3, dtype: int64
```

While *.loc* and *.iloc* are reliable for reading and updating rows, columns, and cells, Pandas provides faster options when you only need a single scalar value. Use **.at** and **.iat** for fast scalar access because they are designed specifically for one cell at a time.

```
import pandas as pd

df = pd.DataFrame(
  data=[['p1', 98.1, 110, 76], ['p2', 78.0, 120, 87], ['p3', 65.0, 129, 77],
['p4', 64.0, 141, 59]],
  columns=['participant', 'hr1', 'hr2', 'hr3']
).set_index('participant')
print(df.iat[3, 2])        # Position-based scalar access (row 3, column 2)
print(df.at['p4', 'hr3'])  # Label-based scalar access (row label p4, column label
hr3)


# Output:
# 59
# 59
```

Common Pitfalls

Two common sources of confusion involve slicing behavior and return types. When slicing rows, **.loc[]** includes both the starting and ending labels, while **.iloc[]** follows standard Python slicing rules and excludes the ending position.

Another common pitfall is confusing Series and DataFrames. Using df['hr3'] returns a Series because a single column is selected, while df[['hr3']] returns a DataFrame because the double brackets indicate a list of columns (even though only one column was included in this example, many could have been included). This distinction becomes important when chaining methods or passing results into other functions.

Indexes

There will also be times when you need to read the index values of a DataFrame, usually the column labels and sometimes the row labels. This often happens when you want to loop through labels and apply the same operation to each column or each row.

```
# Option 1: Cast the DataFrame to a Python list (keeps only the column labels)
columns = list(df)
columns


# Output:
# ['hr1', 'hr2', 'hr3']
```

Notice that if you set *participant* as the row index, it will not appear in the list of column labels because it is no longer a column.

Although casting a DataFrame to a list works, Pandas provides a direct attribute for column labels:

```
# Option 2: Use the DataFrame.columns attribute
columns = df.columns
columns


# Output:
# Index(['hr1', 'hr2', 'hr3'], dtype='object')
```

This approach returns an *Index* object, which includes useful metadata and works naturally with other Pandas operations. There is a similar attribute for returning row labels:

```
rows = df.index
rows


# Output:
# Index(['p1', 'p2', 'p3', 'p4'], dtype='object')
```

You can loop through these labels and print them out one at a time.

```
# Loop through the column labels; print them out
for col in columns:
  print(col)


# Output:
# hr1
# hr2
# hr3
```

In a loop, the variable name (such as *col*) refers to each value in the iterable object. In this case, it refers to each column label. Because *col* is a column label, you can also use it to access the corresponding column in the DataFrame.

```
# Loop through the columns of df; each iteration variable is a column label
for col in df:
  print(f&quot;Column: {col}&quot;)
  print(df[col])
  print()


# Output:
# Column: hr1
# p1    98.1
# p2    78.0
# p3    65.0
# p4    64.0
# Name: hr1, dtype: float64
#
# Column: hr2
# p1    110
# p2    120
# p3    129
# p4    141
# Name: hr2, dtype: int64
#
# Column: hr3
# p1    76
# p2    87
# p3    77
# p4    59
# Name: hr3, dtype: int64
```

Notice a few things. First, you did not need to manually create a list of column labels in order to loop through them; iterating over *df* iterates over the column labels automatically. Second, the loop variable (*col*) is the column label, which can be used to select the entire column with *df[col]*. This looping pattern is a common way to apply the same logic to many columns.

2.4Modifying DataFrames

Add Rows

Adding values to a DataFrame works differently for rows versus columns. To add rows one at a time, you will typically use *.loc* (label-based). The key idea

is that *.loc* can create a new row label if it does not already exist, while *.iloc* can only assign to rows that already exist by position.

```
import pandas as pd

df = pd.DataFrame(columns=['participant', 'hr1', 'hr2', 'hr3'])
df.set_index('participant', inplace=True)
p1_list = [98.1, 110, 76]
p2_list = [78.0, 120, 87]
p3_list = [65.0, 129, 77]
p4_list = [64.0, 141, 59]
df.loc['p1'] = p1_list
df.loc['p2'] = p2_list
df.loc['p3'] = p3_list
df.loc['p4'] = p4_list

# ...or, add the lists directly like so:
df.loc['p1'] = [98.1, 110, 76]
df.loc['p2'] = [78.0, 120, 87]
df.loc['p3'] = [65.0, 129, 77]
df.loc['p4'] = [64.0, 141, 59]
df
```

participant	hr1	hr2	hr3
p1	98.0	110.0	76.0
p2	78.0	120.0	87.0
p3	65.0	129.0	77.0
p4	64.0	141.0	59.0

Notice that we added rows by referring to the row label (for example, *df.loc['p1']*). Those labels did not exist until we used them, so Pandas created new rows automatically.

This works differently with *.iloc*. Because *.iloc* is position-based, it cannot create new rows. To assign with *.iloc*, you must predefine the number of rows first.

```
import pandas as pd

df = pd.DataFrame(index=[0, 1, 2, 3], columns=['participant', 'hr1', 'hr2',
'hr3'])
df.set_index('participant', inplace=True)
df.iloc[0] = [98.1, 110, 76]
df.iloc[1] = [78.0, 120, 87]
df.iloc[2] = [65.0, 129, 77]
```

```
df.iloc[3] = [64.0, 141, 59]
df
```

	hr1	hr2	hr3
participant			
NaN	98.1	110	76
NaN	78	120	87
NaN	65	129	77
NaN	64	141	59

Notice that the row index labels are missing (NaN). That happened because we set *participant* as the index before we filled it with values. When you plan to assign with *.iloc*, it is usually easier to fill the *participant* column first, then set it as the index afterward.

A common Python function used for predefining row positions is *range()*, which produces numbers from 0 to *n* - 1.

```
import pandas as pd

df = pd.DataFrame(index=range(4), columns=['participant', 'hr1', 'hr2', 'hr3'])
df.iloc[0] = ['p1', 98.1, 110, 76]
df.iloc[1] = ['p2', 78.0, 120, 87]
df.iloc[2] = ['p3', 65.0, 129, 77]
df.iloc[3] = ['p4', 64.0, 141, 59]
df.set_index('participant', inplace=True)
df
```

participant	hr1	hr2	hr3
p1	98.0	110.0	76.0
p2	78.0	120.0	87.0
p3	65.0	129.0	77.0
p4	64.0	141.0	59.0

Add Columns

Adding columns is usually straightforward, but you should understand how Pandas aligns values. When you assign a plain Python list, Pandas matches values by row position. When you assign a Series or join another DataFrame, Pandas can align values by index label.

The simplest approach is adding native Python lists. The `.insert()` A method of the Pandas DataFrame object that inserts a column into a specific position. method is useful when you care about column order.

```
import pandas as pd

df = pd.DataFrame(
  data=[['p1', 98.1, 110, 76], ['p2', 78, 120, 87], ['p3', 65, 129, 77], ['p4',
64, 141, 59]],
  columns=['participant', 'hr1', 'hr2', 'hr3']
)

df.set_index('participant', inplace=True)

# These are native Python lists (position-based alignment)
hr4 = [81, 84, 75, 64]
age = [25, 49, 51, 18]
df['hr4'] = hr4           # Add to the end of the columns
df.insert(0, 'Age', age)  # Insert at position 0 (position, column name, values)
df
```

	Age	hr1	hr2	hr3	hr4
p1	25	98.1	110	76	81
p2	49	78.0	120	87	84
p3	51	65.0	129	77	75
p4	18	64.0	141	59	64

Lists are easy to add, but they have a limitation: Pandas cannot match list values to specific row labels. It simply assigns in order. When you need index-based alignment (similar to how a database matches rows), use a Series or another DataFrame and combine them using *.join()* or *.merge()*.

The `.join()` A method for combining DataFrames or Series using their index labels. method is designed for index-based joins.

```
import pandas as pd

df = pd.DataFrame(
  {'hr1':[98.1, 78, 65, 64], 'hr2':[110, 120, 129, 141], 'hr3':[76, 87, 77, 59]},
  index=['p1', 'p2', 'p3', 'p4']
)
```

```
        # This is a Pandas Series; notice there are five records instead of four
        age = pd.Series([25, 49, 51, 18, 36], name='Age', index=['p1', 'p2', 'p3', 'p4',
'p5'])

        # This is a Pandas DataFrame; notice the index is in a different sort order
        df2 = pd.DataFrame(
          {'hr4':[81, 84, 75, 64, 72], 'hr5':[88, 92, 79, 67, 80]},
          index=['p4', 'p1', 'p3', 'p5', 'p2']
        )

        df = df.join(age, how='outer')    # Series and DataFrames need to be added to a
new
        df = df.join(df2, how='inner')    # version of the DataFrame--even if it's the
same name
        df
```

	hr1	hr2	hr3	Age	hr4	hr5
p1	98.1	110.0	76.0	25	84	92
p2	78.0	120.0	87.0	49	72	80
p3	65.0	129.0	77.0	51	75	79
p4	64.0	141.0	59.0	18	81	88
p5	NaN	NaN	NaN	36	64	67

Index-based joins can handle mismatched row counts and different sort orders because Pandas matches on index labels (not row position). The tradeoff is that *.join()* does not let you choose where the new columns appear; it appends them to the right.

Sometimes you need to combine datasets using key columns (not indexes) and duplicate values across multiple related rows (similar to a relational database). That is what .merge() A relational-style merge that matches rows using one or more key columns. is designed for.

For example, suppose heart rate data were recorded in a long format (one row per measurement) rather than a wide format (one row per participant):

```
import pandas as pd

hr_df = pd.DataFrame(
  {
    'participant': ['p1', 'p2', 'p3', 'p4', 'p5', 'p2', 'p3', 'p4'],
    'hr': [98.1, 78, 65, 64, 76, 87, 77, 59]
  }
)

age_df = pd.DataFrame(
  {
    'participant': ['p1', 'p2', 'p3', 'p4', 'p5'],
    'Age': [25, 49, 51, 18, 36],
    'Gender': ['m', 'f', 'f', 'm', 'f']
  }
)

print(hr_df)
print(age_df)
age_df.merge(hr_df, how='inner', on='participant')


# Output:
#   participant    hr
# 0          p1  98.1
# 1          p2  78.0
# 2          p3  65.0
# 3          p4  64.0
# 4          p5  76.0
# 5          p2  87.0
# 6          p3  77.0
# 7          p4  59.0
#   participant  Age Gender
# 0          p1   25      m
# 1          p2   49      f
# 2          p3   51      f
# 3          p4   18      m
# 4          p5   36      f
```

	age	gender	participant	hr
0	25	m	p1	98.1
1	49	f	p2	78.0
2	49	f	p2	87.0
3	51	f	p3	65.0
4	51	f	p3	77.0
5	18	m	p4	64.0
6	18	m	p4	59.0
7	36	f	p5	76.0

In this example, *.merge()* duplicates each participant's demographic information across multiple heart rate measurements. If your key column names are different, you can specify them with *left_on* and *right_on*.

Summary of Techniques for Combining Two DataFrames

Here are common ways to combine DataFrames and when each one tends to be the best choice:

- **Join** combines DataFrames by matching index labels. It is fast and simple, but assumes you are joining on indexes.

```
joined_df = df1.set_index('key').join(df2.set_index('key'), how='left')
```

- **Merge** combines DataFrames by matching one or more key columns (similar to SQL joins). It supports inner, outer, left, and right joins and is

the most flexible option.

```
merged_df = pd.merge(df1, df2, how='inner', on='key')
```

- **Concat** stacks DataFrames along an axis (rows or columns). Use *axis=0* to stack rows and *axis=1* to stack columns. It supports *join='inner'* (intersection) and *join='outer'* (union).

```
concatenated_df = pd.concat([df1, df2], axis=1, join='outer')
```

- You may see **append()** in older code. It was deprecated and removed in modern Pandas. Use *pd.concat()* with *axis=0* to append rows instead:

```
# Old (removed): df1.append(df2, ignore_index=True)
# Modern equivalent:
appended_df = pd.concat([df1, df2], axis=0, ignore_index=True)
```

A simple rule of thumb is: if you want to join by index, use *join()*; if you want to join by key columns, use *merge()*.

Edit/Update

Updating values works almost exactly like adding values. The main difference is that you assign to labels or positions that already exist. When you update an entire row or column, you must provide a value for every field in that row or column.

```
import pandas as pd

df = pd.DataFrame(
  {'hr1':[98.1, 78, 65, 64], 'hr2':[110, 120, 129, 141], 'hr3':[76, 87, 77, 59]},
  index=['p1', 'p2', 'p3', 'p4']
)

df.loc['p1'] = [99, 111, 77]
```

```
df.iloc[0] = [99, 111, 77]
df
```

	hr1	hr2	hr3
p1	99.0	111	77
p2	78.0	120	87
p3	65.0	129	77
p4	64.0	141	59

Updating a column works the same way, either by label or by position:

```
import pandas as pd

df = pd.DataFrame(
  {'hr1':[98.1, 78, 65, 64], 'hr2':[110, 120, 129, 141], 'hr3':[76, 87, 77, 59]},
  index=['p1', 'p2', 'p3', 'p4']
)

df['hr1'] = [99.1, 78.4, 76.6, 63.9]
df.iloc[:, 0] = [99.1, 78.4, 76.6, 63.9]
df
```

	hr1	hr2	hr3
p1	99.1	110	76
p2	78.4	120	87
p3	76.6	129	77
p4	63.9	141	59

To update a single cell value, you can use *.loc* and *.iloc*, but it is more efficient to use **.at** and **.iat** because they are designed for one scalar value at a time.

```
df.at['p1', 'hr1'] = 99
df.iat[1, 0] = 78
df
```

	hr1	hr2	hr3
p1	99.0	110	76
p2	78.0	120	87
p3	76.6	129	77
p4	63.9	141	59

Delete

Deleting rows and columns is usually done with *.drop()*. By default, Pandas assumes you want to drop rows (*axis=0*). To drop columns, use *axis=1*.

```
import pandas as pd

df = pd.DataFrame(
  {'hr1':[98.1, 78, 65, 64], 'hr2':[110, 120, 129, 141], 'hr3':[76, 87, 77, 59]},
  index=['p1', 'p2', 'p3', 'p4']
)

df.drop(['p1', 'p3'], inplace=True)
df.drop(['hr1'], axis=1, inplace=True)
df
```

	hr2	hr3
p2	120	87
p4	141	59

You can also drop rows or columns by position by referencing *df.index* or *df.columns* and then selecting elements by number:

```
import pandas as pd

df = pd.DataFrame(
  {'hr1':[98.1, 78, 65, 64], 'hr2':[110, 120, 129, 141], 'hr3':[76, 87, 77, 59]},
  index=['p1', 'p2', 'p3', 'p4']
)

df.drop(df.index[2], inplace=True)
df.drop(df.columns[1], axis=1, inplace=True)
df
```

	hr1	hr3
p1	98.1	76
p2	78.0	87
p4	64.0	59

You can drop a list of positions at once by passing multiple indices:

```
import pandas as pd

df = pd.DataFrame(
  {'hr1':[98.1, 78, 65, 64], 'hr2':[110, 120, 129, 141], 'hr3':[76, 87, 77, 59]},
  index=['p1', 'p2', 'p3', 'p4']
)
```

```
df.drop(df.index[[1, 2]], inplace=True)
df
```

	hr1	hr2	hr3
p1	98.1	110	76
p4	64.0	141	59

Negative index positions count from the end of the index list, which can be useful when you want to drop from the bottom up:

```
import pandas as pd

df = pd.DataFrame(
  {'hr1':[98.1, 78, 65, 64], 'hr2':[110, 120, 129, 141], 'hr3':[76, 87, 77, 59]},
  index=['p1', 'p2', 'p3', 'p4']
)

df.drop(df.index[[-2]], inplace=True)
df
```

	hr1	hr2	hr3
p1	98.1	110	76
p2	78.0	120	87
p4	64.0	141	59

2.5Filtering

Often, we do not want to delete rows unless they are truly junk data. In most analysis workflows, we prefer to create a new, filtered version of a

DataFrame while leaving the original unchanged. Pandas makes filtering fast, readable, and (once you learn the patterns) safer than looping through rows.

A common filtering pattern in Pandas looks like this:

Filter by Row

Pandas filters rows using this syntax:

```
df[conditional]
```

The *conditional* is a boolean expression that evaluates to *True* or *False* for each row. Rows where the conditional is *True* are kept; rows where it is *False* are removed from the result.

You will often refer to a column inside the conditional. You may see two styles:

```
df['column_label']   # Recommended (works with any valid column label)
df.column_label      # Convenient, but only works for some column names
```

In this book, we will prefer *df['column_label']* because it is more reliable (for example, it works even if a column name contains spaces or conflicts with a DataFrame method name).

Here is an example with a few more columns.

```
import pandas as pd

df = pd.DataFrame(
  {
    'age': [29, 55, 65, 18],
    'gender': ['male', 'female', 'male', 'female'],
    'hr1': [98.1, 78, 65, 64],
```

```
    'hr2': [110, 120, 129, 141],
    'hr3': [76, 87, 77, 59]
  },
  index=['p1', 'p2', 'p3', 'p4']
)

filtered_df = df[df['age'] &gt; 30]  # Evaluated as True/False for each row
filtered_df
```

	age	gender	hr1	hr2	hr3
p2	55	female	78.0	120	87
p3	65	male	65.0	129	77

Notice that the entire row is returned if the condition is *True* for that row. Try filtering out all males and ignoring age.

```
import pandas as pd

df = pd.DataFrame(
  {
    'age': [29, 55, 65, 18],
    'gender': ['male', 'female', 'male', 'female'],
    'hr1': [98.1, 78, 65, 64],
    'hr2': [110, 120, 129, 141],
    'hr3': [76, 87, 77, 59]
  },
  index=['p1', 'p2', 'p3', 'p4']
)

filtered_df = df[df['gender'] == 'female']
filtered_df
```

	age	gender	hr1	hr2	hr3
p2	55	female	78.0	120	87
p4	18	female	64.0	141	59

Now return all records of females over 30 years old.

```
import pandas as pd

df = pd.DataFrame(
  {
    'age': [29, 55, 65, 18],
    'gender': ['male', 'female', 'male', 'female'],
    'hr1': [98.1, 78, 65, 64],
    'hr2': [110, 120, 129, 141],
    'hr3': [76, 87, 77, 59]
  },
  index=['p1', 'p2', 'p3', 'p4']
)

filtered_df = df[(df['gender'] == 'female') &amp; (df['age'] &gt; 30)]
filtered_df
```

	age	gender	hr1	hr2	hr3
p2	55	female	78.0	120	87

Pandas is different from native Python here: you must include parentheses around each condition, use & instead of *and*, and use | instead of *or*. Now return all records where *hr1* or *hr3* are over 90.

```
import pandas as pd

df = pd.DataFrame(
  {
    'age': [29, 55, 65, 18],
    'gender': ['male', 'female', 'male', 'female'],
    'hr1': [98.1, 78, 65, 64],
    'hr2': [110, 120, 129, 141],
    'hr3': [76, 87, 77, 59]
  },
  index=['p1', 'p2', 'p3', 'p4']
)

filtered_df = df[(df['hr1'] &gt; 90) | (df['hr3'] &gt; 90)]
filtered_df
```

	age	gender	hr1	hr2	hr3
p1	29	male	98.1	110	76

Filtering is also much faster than looping through every row, because Pandas performs these operations in optimized, vectorized code.

Common pitfall: SettingWithCopyWarning. You may eventually see a warning that looks like: *SettingWithCopyWarning*. This usually happens when you filter a DataFrame and then try to modify the filtered result. The reason is that Pandas cannot always tell whether your filtered object is a true copy (safe to modify) or a view into the original DataFrame (where a change might or might not affect the original). To avoid confusion, use one of these safe patterns: (1) if you intend to modify the original DataFrame, do the assignment on the original using *df.loc[condition, 'column'] = value*; or (2) if you intend to create a new, independent DataFrame that you will modify, make an explicit copy using *filtered_df = df[condition].copy()*.

Filter by Column

Pandas also allows you to select a subset of columns by name using this syntax (note the double brackets):

```
df[[column_list]]
```

An example:

```
import pandas as pd

df = pd.DataFrame(
  {
    'age': [29, 55, 65, 18],
    'gender': ['male', 'female', 'male', 'female'],
    'hr1': [98.1, 78, 65, 64],
    'hr2': [110, 120, 129, 141],
    'hr3': [76, 87, 77, 59]
  },
  index=['p1', 'p2', 'p3', 'p4']
)

filtered_df = df[['age', 'gender']]  # A list of column labels inside brackets
filtered_df
```

	age	gender
p1	29	male
p2	55	female
p3	65	male
p4	18	female

DataFrame.filter()

Pandas also includes a built-in `.filter()` A method for filtering by row or column labels, including partial matches (like) and regular expressions (regex). method. This example uses the *like* parameter to return any column label that contains the letters *hr*:

```
import pandas as pd

df = pd.DataFrame(
  {
    'age': [29, 55, 65, 18],
    'gender': ['male', 'female', 'male', 'female'],
    'hr1': [98.1, 78, 65, 64],
```

```
            'hr2': [110, 120, 129, 141],
            'hr3': [76, 87, 77, 59]
          },
          index=['p1', 'p2', 'p3', 'p4']
        )

        filtered_df = df.filter(like='hr', axis=1)  # Filter column labels that include
'hr'
        filtered_df
```

	hr1	hr2	hr3
p1	98.1	110	76
p2	78.0	120	87
p3	65.0	129	77
p4	64.0	141	59

2.6Sorting

Like native Python lists and dictionaries, DataFrames can also `sort` An action that changes the order of records in a dataset based on rules. by one or more fields. In Pandas, the two most common sorting methods are `sort_index()` Pandas DataFrame method used to sort objects by labels along either rows or columns. and `sort_values()` Pandas DataFrame method used to sort objects by the actual data values in one or more columns (or rows)..

A simple way to remember the difference is: **sort_index()** sorts labels (index labels or column labels), while **sort_values()** sorts actual numeric or text values inside the table.

Sorting by index labels. The *sort_index()* method sorts whatever the current index is. If you never set a labeled index, Pandas uses a numeric RangeIndex by default. But if you set a labeled row index (for example, participant IDs)

and the labels are out of order, *sort_index()* will reorder rows by those labels. You can also reverse the order:

```
import pandas as pd

df = pd.DataFrame(
  {
    'age': [29, 55, 65, 18],
    'gender': ['male', 'female', 'male', 'female'],
    'hr1': [98.1, 78, 65, 64],
    'hr2': [110, 120, 129, 141],
    'hr3': [76, 87, 77, 59]
  },
  index=['p1', 'p2', 'p3', 'p4']
)

df = df.sort_index(ascending=False)
df
```

	age	gender	hr1	hr2	hr3
p4	18	female	64.0	141	59
p3	65	male	65.0	129	77
p2	55	female	78.0	120	87
p1	29	male	98.1	110	76

You can use *sort_index()* to sort column labels as well by setting *axis=1*:

```
import pandas as pd

df = pd.DataFrame(
  {
    'age': [29, 55, 65, 18],
    'gender': ['male', 'female', 'male', 'female'],
    'hr1': [98.1, 78, 65, 64],
    'hr2': [110, 120, 129, 141],
    'hr3': [76, 87, 77, 59]
  },
  index=['p1', 'p2', 'p3', 'p4']
```

```
)

df = df.sort_index(axis=1, ascending=False)
df
```

	hr3	hr2	hr1	gender	age
p1	76	110	98.1	male	29
p2	87	120	78.0	female	55
p3	77	129	65.0	male	65
p4	59	141	64.0	female	18

Sorting by values. Use *sort_values()* to sort rows based on the values in one or more columns. Here, we sort people by age (smallest to largest):

```
import pandas as pd

df = pd.DataFrame(
  {
    'age': [29, 55, 65, 18],
    'gender': ['male', 'female', 'male', 'female'],
    'hr1': [98.1, 78, 65, 64],
    'hr2': [110, 120, 129, 141],
    'hr3': [76, 87, 77, 59]
  },
  index=['p1', 'p2', 'p3', 'p4']
)

df = df.sort_values(by=['age'])
df
```

	age	gender	hr1	hr2	hr3
p4	18	female	64.0	141	59
p1	29	male	98.1	110	76
p2	55	female	78.0	120	87
p3	65	male	65.0	129	77

You can also sort by multiple levels (multiple columns). The first column listed becomes the primary sort key, and the next column becomes a tie-breaker:

```
import pandas as pd

df = pd.DataFrame(
  {
    'age': [29, 55, 65, 18],
    'gender': ['male', 'female', 'male', 'female'],
    'hr1': [98.1, 78, 65, 64],
    'hr2': [110, 120, 129, 141],
    'hr3': [76, 87, 77, 59]
  },
  index=['p1', 'p2', 'p3', 'p4']
)

df = df.sort_values(by=['gender', 'age'])
df
```

	age	gender	hr1	hr2	hr3
p4	18	female	64.0	141	59
p2	55	female	78.0	120	87
p1	29	male	98.1	110	76
p3	65	male	65.0	129	77

Optional note: if you ever need different sort directions for different columns (for example, gender ascending and age descending), *sort_values()* accepts a matching list such as *ascending=[True, False]*. You will see this pattern often in real analytics work.

2.7Practice

Test your understanding of Pandas DataFrames by completing the practice problems below. If you get stuck, focus on (1) how the DataFrame is created, (2) how columns are added or selected, and (3) how filtering and sorting work.

Practice #1a: Favorites
Import the Pandas library and create a new DataFrame named *df*. The DataFrame should have three columns: **Name**, **Favorite Food**, and **Favorite Drink**. Add three people (three rows) to the DataFrame.

Practice #1b: Add Column from List
Add a new column named **Age** to your DataFrame. Create a Python list of ages in the same row order as the people in *df*, then add it as a new column.

Next, create a new DataFrame that includes only the **Name** and **Age** columns (filter out the other columns).

Practice #1c: Add Column and Update from Loop

Create a new column named **BornBefore2000** and fill it with *True* or *False* for each person. Assume anyone age *20 or under* was born in 2000 or later, and anyone age *over 20* was born before 2000. Your solution should work no matter how many rows are in the DataFrame, so you must loop through the records.

Practice #2a: Create Movie Streaming DataFrame

You are building a streaming service and just acquired your first five movies. Create a DataFrame with four columns: **Movie Name**, **Rating**, **Run Time**, and **Main Genre**. Use the dataset below.

```
[['Remember the Titans','PG',113, 'Sport'], ['Forrest Gump', 'PG-13',142,'Drama'], ['Inception','PG-13',148,'Sci-Fi'], ['The Proposal','PG-13',108,'Romance'], ['Dumb &amp;amp; Dumber','PG-13',108,'Comedy']]
```

Practice #2b: Add Column

Add a new column named **Year** that stores each movie's release year. Use the values below, and make sure the years align with the correct movies.

```
# Release years
# Remember the Titans: 2000
# Forrest Gump: 1994
# Inception: 2010
# The Proposal: 2009
# Dumb &amp;amp; Dumber: 1994
```

Practice #2c: Delete

Warner Bros. Pictures has pulled its movies from streaming platforms. Remove *Inception* from the movie library.

Practice #2d: Read

Customers want to see what genres are available. Print the main genre for each movie (one genre per movie).

Practice #2e: Filter

A potential partner only supports movies shorter than two hours. Create a new DataFrame (or list) containing all movies that are **two hours or longer** and would need to be removed. (Two hours is 120 minutes.)

Practice #2f: Sort

Sort your movie library by **Run Time** (shortest to longest). If two movies have the same run time, sort those ties by **Year** (oldest to newest).

2.8Assignment

The assignment for this chapter is combined with the next chapter. You can proceed to the next chapter.